

🔥 START CHECKBOX 6 — ASYNC + PARSING BOUNDARY

(This box saves you from lifelong over-engineering. Read carefully.)

We'll do this **properly**:

1. Teach (one core idea at a time)
2. Hands-on thinking
3. 3 thinking exercises
4. Debug a wrong pattern
5. Lock the rule forever

6 ASYNC + PARSING BOUNDARY

🧠 CORE IDEA (Feynman, 2 lines)

Async = waiting for outside world (network).

Parsing = CPU work → keep it sync.

Or in one brutal rule:

Network waits → **async**

CPU thinks → **sync**

? WHY THIS CHECKBOX EXISTS (VERY IMPORTANT)

Most people learn async and then do this 🙄

- ✗ “Let’s make EVERYTHING async”
- ✗ “BeautifulSoup ko bhi async bana dete hain”
- ✗ “Har function async def hona chahiye”

👉 Result:

- Slow code

- Complex code
- Bugs
- No real speed gain

Checkbox 6 exists to stop this madness.



MENTAL MODEL (LOCK THIS)

Two different worlds



WORLD 1 — Network (Async)

- HTTP request
- Waiting on server
- I/O bound
- Python sits idle → async helps



WORLD 2 — Parsing (Sync)

- BeautifulSoup
- Regex
- String processing
- CPU bound
- No waiting → async useless



REAL-LIFE ANALOGY



Restaurant analogy

- **Async fetch** = waiter waiting for food 🍽️
- **Parsing** = chef cutting vegetables 🍴

Question:

“Should chef wait asynchronously to cut vegetables?”

✗ No.

Chef just works.

Same with BeautifulSoup.



RULES YOU MUST NEVER BREAK



NEVER DO THIS

```
async def parse_html(html):  
    soup = BeautifulSoup(html, "lxml")  
    ...
```

Why ✗?

- No waiting involved
- Async gives **zero benefit**
- Adds mental + code overhead



ALWAYS DO THIS

```
def parse_html(html):  
    soup = BeautifulSoup(html, "lxml")  
    ...
```

Call it **normally**, even from async code.



CORRECT SYSTEM ARCHITECTURE

```
async main()
└─ async fetch_html() 🌐 (network)
    ↓
    HTML string
    ↓
    parse_html() 🗨️ (sync)
    ↓
    structured data
```

Clean boundary. No mixing.



HANDS-ON PRACTICE (THINKING)

Scenario

You scrape 20 pages.

Each page:

- Network wait = 500 ms
- Parsing time = 5 ms

? Question:

Where is the real bottleneck?

👉 **Network**, not parsing.

So:

- Make network async ✅
- Parsing sync is fine ✅



THINKING EXERCISES (3)



Exercise 1 — Wrong instinct

Why does making BeautifulSoup async **not** make parsing faster?

(Hint: async ≠ parallel CPU)



Exercise 2 — Separation

Where should this code live?

```
title = soup.find("h1").text
```

- fetch_html?
- parse_html?
- main?



WHY?



Exercise 3 — Scaling fear

“If parsing is sync, won’t it slow down my async scraper?”

Correct answer:

- Parsing is tiny
- Network dominates
- Premature optimization is poison



DEBUG THIS CODE (VERY COMMON MISTAKE)

```
async def fetch_and_parse(session, url, sem):
    async with sem:
        async with session.get(url) as response:
            html = await response.text()
            soup = BeautifulSoup(html, "lxml")
            return soup.find("h1").text
```

? What's wrong here?

Answer:

- Network + parsing tightly coupled ❌
- No reusable parse stage
- Hard to test
- Hard to extend
- Violates separation of concerns



CORRECT VERSION (MENTAL, NOT FULL CODE)

```
async def fetch_html(...):
    return html

def parse_html(html):
    return data
```

And in `main()` :

```
html = await fetch_html(...)
data = parse_html(html)
```

GOLDEN RULE (WRITE THIS SOMEWHERE)

Async is a tool, not a religion.

Use it **only** where waiting exists.

WHAT YOU CAN NOW CONFIDENTLY SAY

- I will **never** **async-ify** BeautifulSoup
- I know where async stops
- I know where sync starts
- My scraper architecture is clean
- I avoid unnecessary complexity

You have completed the **ENTIRE CORE** of Stage 2.

You are capable of:

- Designing async scrapers
- Controlling speed
- Handling failures
- Avoiding over-engineering

NEXT OPTIONS (YOUR CALL)

We can now:

- 1 Build a full mini scraper (fetch + parse + save)**

- 2 Move to **rate-limiting + polite crawling**
- 3 Start **Stage 3 (Data parsing & extraction patterns)**

Just tell me what you want next.